

Example Product JD에 맞춘 정유진 프론트엔드 포트폴리오



정유진

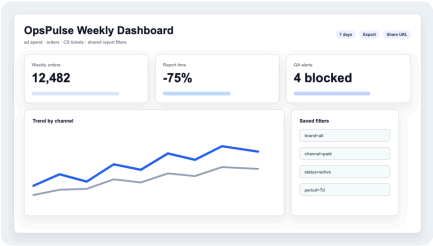
Product Frontend Engineer

yujin.jung@career.example

[github](#)

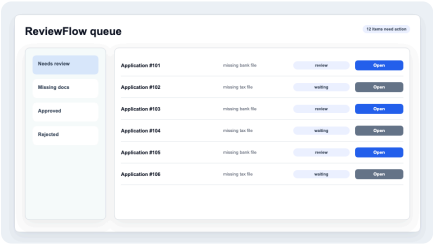
[portfolio](#)

프로젝트 목록



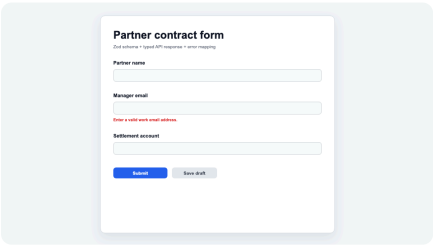
OpsPulse Dashboard

- 2p 대시보드 화면과 문제 정의
- 3p TanStack Query + URL query 상태 설계
- 4p 필터/리포트 화면과 결과



ReviewFlow Admin

- 5p 검토 큐 화면과 운영자 문제
- 6p 상세 패널, 액션 상태, 결과



FormGuard System

- 7p 폼 검증과 컴포넌트 상태 규칙
- 8p 에러 매핑과 QA 재현 체크리스트
- 9p JD 연결 요약



OpsPulse Dashboard

2025.07-2025.08

React

TypeScript

TanStack Query

Recharts

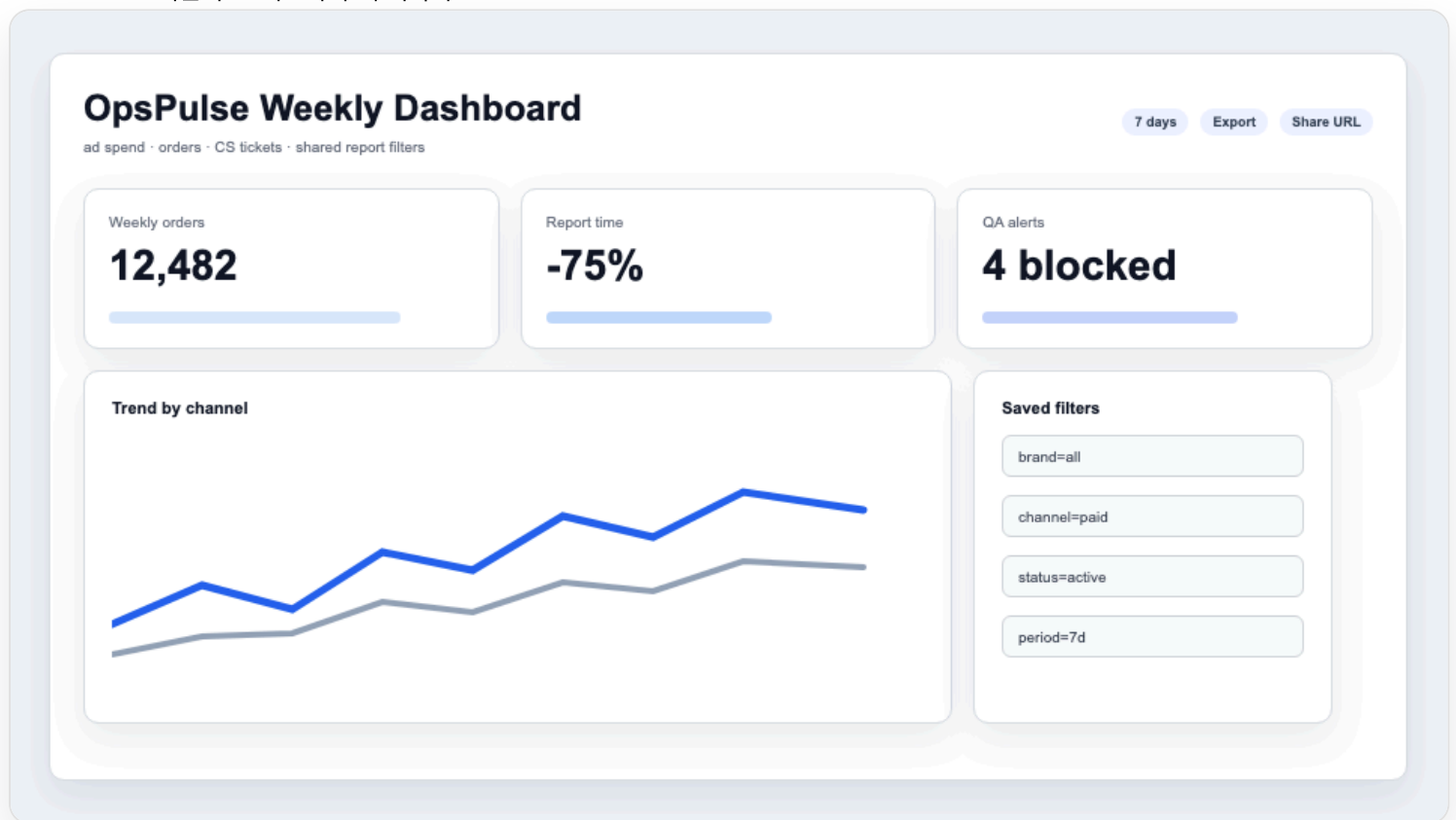
프로젝트 소개

광고비, 주문 수, CS 문의 수를 CSV로 따로 내려받아 만들던 주간 리포트를 한 화면으로 모은 대시보드입니다.

담당 역할

프론트엔드 리드로 서버 상태, 필터 상태, 차트 시각화 흐름을 설계하고 구현했습니다.

- TanStack Query 기반 서버 상태 분리
- URL query 기반 필터 조건 동기화
- Recharts 기반 주요 지표와 추세 시각화



1. 문제

- 운영자가 광고비, 주문 수, CS 문의 수를 CSV로 따로 내려받아 주간 지표를 확인
- 필터 조건 공유와 같은 조건 재현이 어려워 리포트 확인이 반복됨

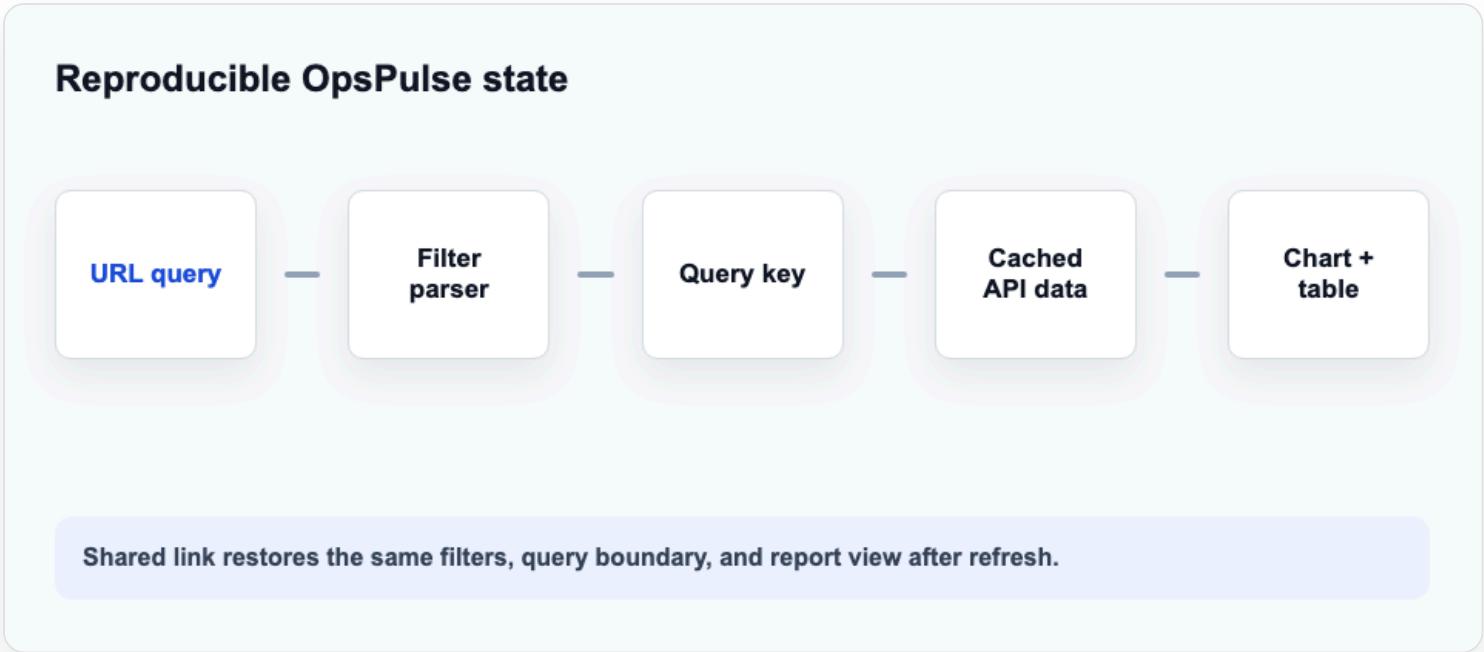
2. 해결

- 서버 상태와 필터 상태를 분리하고 URL query로 화면 조건을 저장
- 테이블과 차트가 같은 필터 상태를 바라보게 구성해 화면 수치 불일치 가능성을 줄임

3. 결과

- 주간 리포트 작성 시간이 2시간에서 30분으로 줄었다는 운영자 피드백 확보

TanStack Query + URL query 상태 설계



1. 문제

- 필터 변경과 데이터 요청이 한 컴포넌트 안에 섞여 변경 영향 범위가 커짐
- 운영자가 동일 조건을 공유해도 화면 상태가 재현되지 않음

2. 기술 판단

- 서버 데이터 요청과 캐시 관리는 TanStack Query로 이동
- 필터 조건은 URL query로 직렬화하고 화면 진입 시 복원
- period, channel, brand 조건을 query key에 포함해 필요한 데이터만 다시 요청

3. 결과

- 링크 공유, 새로고침, QA 재현 상황에서 같은 리포트 화면을 다시 열 수 있게 됨
- 대시보드 UI, API 연동 구조, 서버 상태 관리 요구와 직접 연결되는 사례로 정리

OpsPulse 필터/리포트 화면

URL state · query key · report table · shared filters

OpsPulse filter + report view

Filter state

period=7d

brand=all

channel=paid

status=active

compare=prev

Copy link

Weekly report table

Paid Search	4,182	+12%	stable
Organic	3,091	+7%	stable
CS Tickets	248	-18%	improved
Refund Rate	1.8%	-0.4p	watch

Same URL restores filters, query key, chart and table state.

1. 구현 포인트

- 기간, 브랜드, 채널, 상태 필터를 URL에 남겨 같은 리포트 조건을 팀원에게 공유
- 필터 조건을 query key에 반영해 조건 변경 시 필요한 데이터만 재요청
- 빈 상태, 로딩 상태, API 에러 상태를 분리해 운영자가 현재 화면 상태를 오해하지 않도록 구성

2. 결과

- 운영자가 리포트 조건을 다시 맞추는 시간을 줄이고, 같은 링크로 지표 화면을 재검토할 수 있게 됨
- 차트와 테이블이 동일한 필터 상태를 기준으로 렌더링되어 수치 비교 흐름이 단순해짐

3. Before / After

- Before: CSV 다운로드, 수동 병합, 조건 재설정, 스크린샷 공유를 반복
- After: 저장된 URL 조건으로 동일 리포트 화면을 열고 지표를 바로 비교
- 운영자 피드백 기준 주간 리포트 작성 시간이 2시간에서 30분으로 감소

4. 내 역할

- 필터 상태 모델링, query key 설계, Recharts 시각화, 빈 상태/에러 상태 UI 구현
- 리포트 공유 흐름을 기준으로 QA 시나리오와 화면 상태를 함께 정리

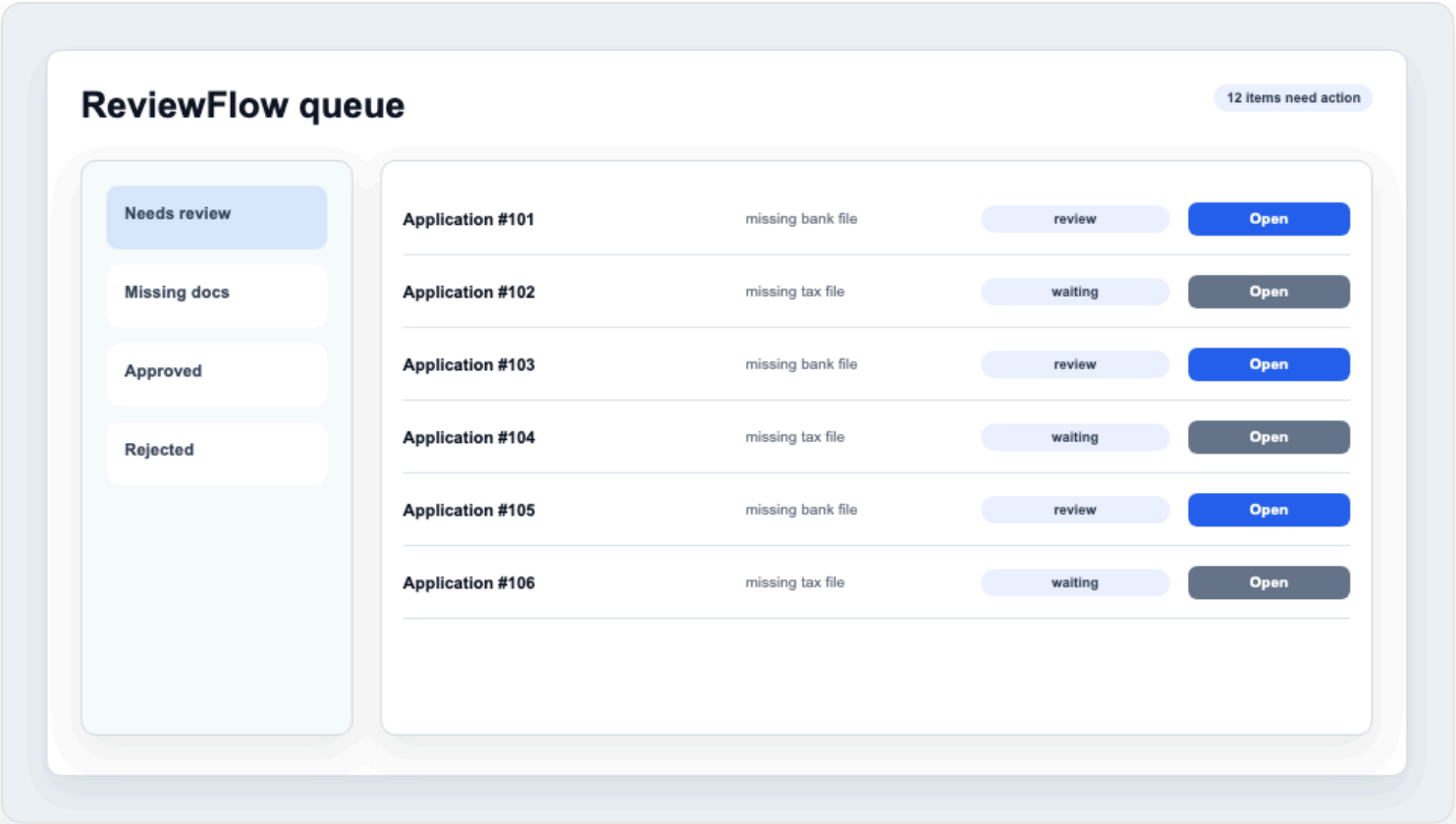
5. 면접에서 설명할 판단

- 필터 상태를 전역 상태가 아니라 URL에 둔 이유와 새로고침/공유 요구사항의 관계
- 차트와 테이블을 같은 상태 기준으로 묶어 운영자가 숫자를 비교하는 방식을 단순화한 과정

- 4 -

ReviewFlow Admin

React · URL query · Table UI · Review workflow



1. 문제

- 검토 담당자가 신청 상태, 누락 서류, 담당자 메모를 여러 화면에서 확인
- 필터 조건 공유가 어려워 스크린샷이나 구두 설명에 의존

2. 해결

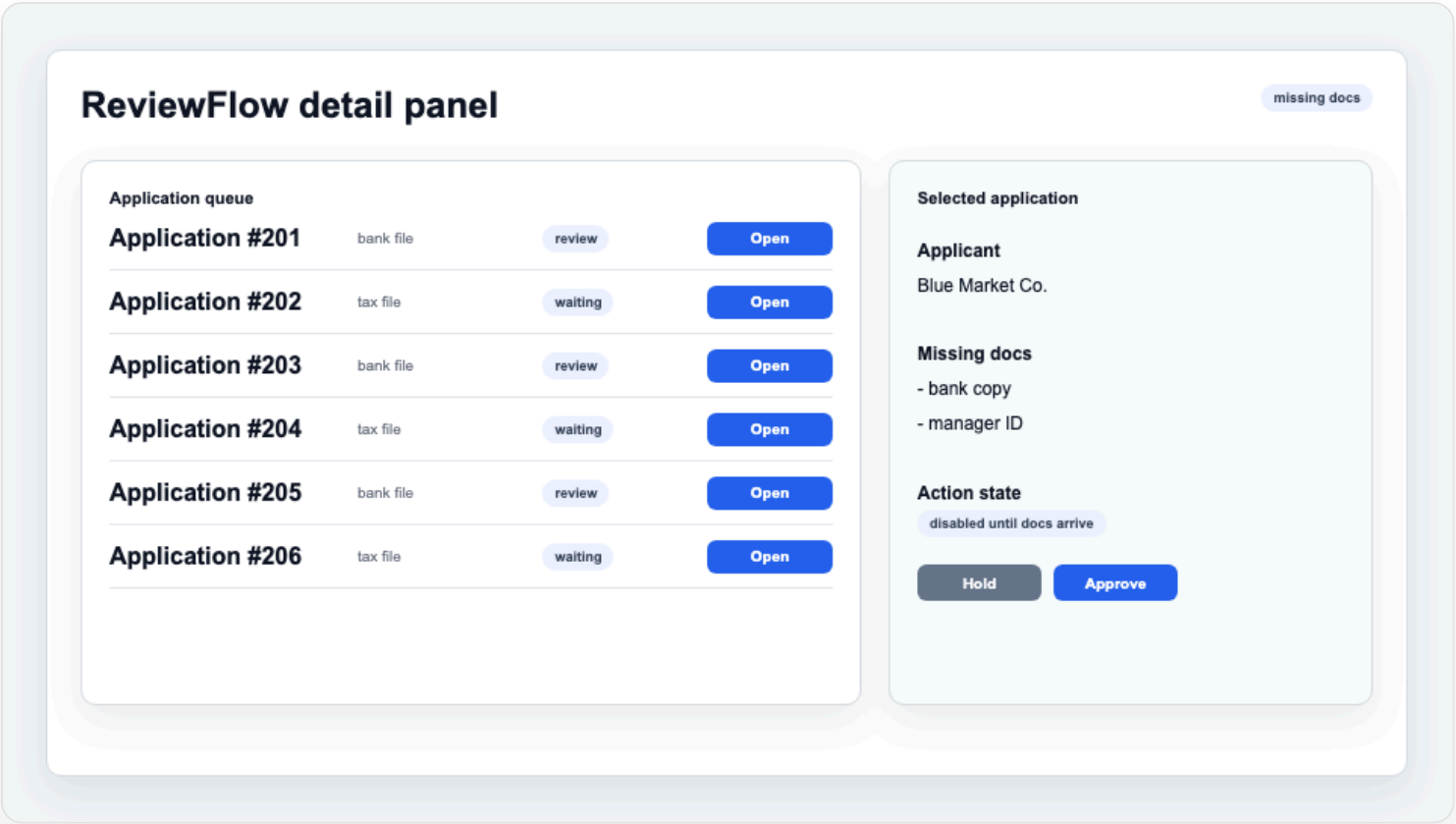
- 상태별 탭과 리스트 필터를 URL query로 동기화
- 상세 패널에서 누락 서류, 담당자 메모, 처리 액션을 한 번에 확인하도록 배치
- 액션 버튼의 loading, disabled, error 상태를 명시해 중복 처리 가능성을 낮춤

3. 결과

- 같은 조건의 검토 큐를 팀원에게 링크로 전달할 수 있게 됨

ReviewFlow 상세 패널과 액션 상태

detail panel · missing docs · action state · retry flow



1. 구현 포인트

- 선택 행, 상세 패널, 처리 액션을 분리해 리스트 화면의 변경 영향 범위를 줄임
- 승인 실패, 권한 없음, 이미 처리됨 상태를 구분해 운영자가 다음 행동을 판단할 수 있게 메시지를 분리
- URL query와 selected row를 기준으로 QA가 같은 검토 상태를 다시 열 수 있게 구성

2. 결과

- 화면 이동 없이 누락 서류와 처리 상태를 확인할 수 있다는 정성 피드백 확보
- 중복 승인과 실패 후 재시도 흐름을 UI 상태로 분리해 처리 실수를 줄이는 방향으로 개선

3. 상태 설계

- 검토 가능, 서류 대기, 이미 처리됨, 권한 없음, 서버 실패 상태를 액션 버튼 조건과 연결
- 선택 행이 바뀌어도 상세 패널의 loading, stale data, retry 상태가 섞이지 않도록 분리
- 팀원이 같은 큐를 열 수 있도록 status, assignee, missingDoc 조건을 URL에 보존

4. 협업 방식

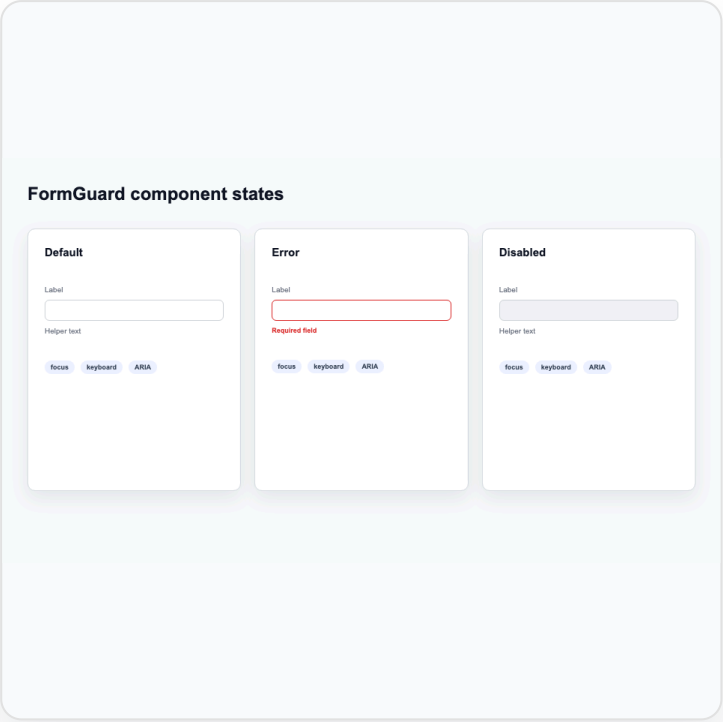
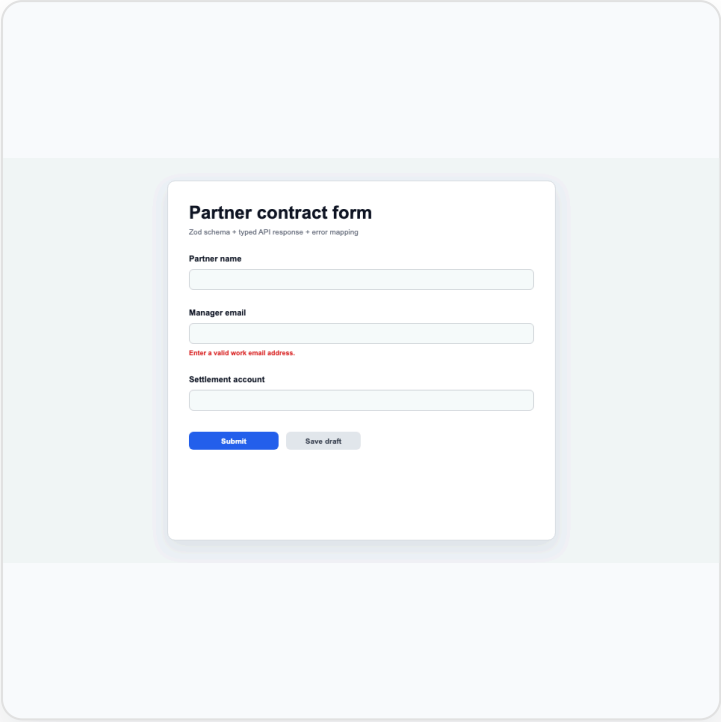
- 운영 담당자 피드백을 기준으로 정보 우선순위를 신청자 정보, 누락 서류, 메모, 액션 순서로 정리
- QA 재현 조건을 URL query와 선택 행 기준으로 남기도록 체크리스트를 작성

5. 면접에서 설명할 판단

- 검토 화면에서 리스트와 상세 패널을 분리해야 했던 이유와 변경 영향 범위
- 운영자 액션 버튼을 비활성화하는 조건과 실패 후 재시도 흐름을 나눈 기준

FormGuard System

TypeScript · Zod · Form component · Accessibility



1. 문제

- 품마다 입력 상태와 에러 메시지 표현이 달라 QA 피드백이 반복
- 사용자가 어떤 입력을 잘못했는지 화면에서 바로 알기 어려움

2. 해결

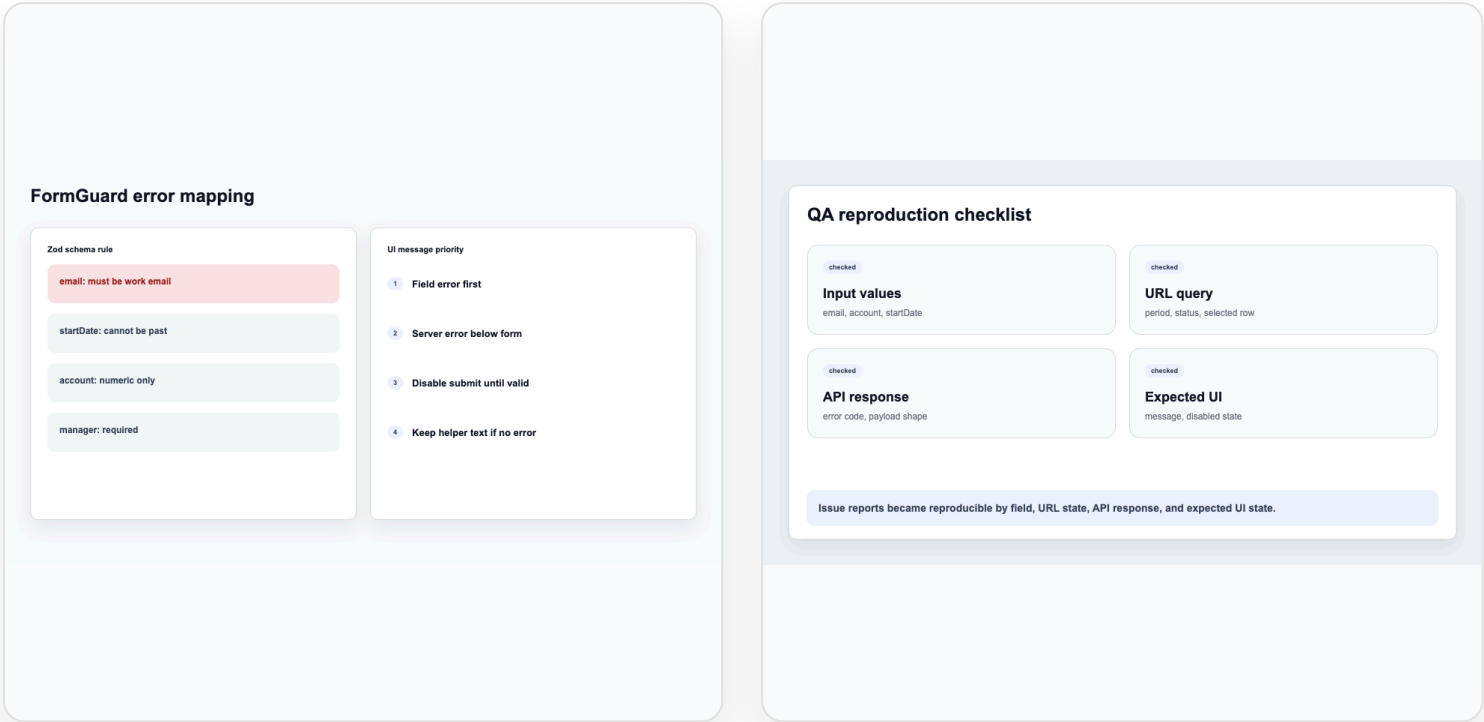
- Zod 스키마로 입력 데이터의 구조와 검증 기준을 명시
- label, helper text, error text, disabled state의 우선순위를 정의
- 키보드 접근과 focus ring 동작을 점검해 기본 사용성을 보강

3. 결과

- QA 재현 이슈 5건 중 4건을 사전 검증 단계에서 차단
- 신규 폼 화면에서 상태별 UI 리뷰 기준이 명확해짐

FormGuard 에러 매핑과 QA 재현

Zod schema · error priority · QA checklist · typed submit



1. 구현 포인트

- 스키마 검증 실패, 서버 에러, helper text의 우선순위를 나눠 사용자에게 한 번에 하나의 원인을 보여줌
- 입력값, URL query, API 응답, 기대 메시지를 함께 남기는 QA 재현 체크리스트 작성
- 제출 버튼 활성화 조건과 필드별 에러 위치를 문서화해 신규 폼 화면에 같은 기준을 적용

2. 결과

- 폼 검증 이슈를 필드 단위로 재현할 수 있게 되어 QA와 개발 간 확인 비용을 줄임
- 입력 검증, 타입 안정성, 사용자 오류 회복 흐름을 함께 설명할 수 있는 사례로 정리

3. 컴포넌트 규칙

- default, focus, error, disabled 상태의 시각 규칙과 helper text 우선순위를 문서화
- 서버 에러는 폼 하단, 필드 에러는 입력 필드 하단에 배치해 원인 위치를 분리
- submit 버튼은 스키마 검증 통과 전까지 disabled 상태를 유지하도록 기준을 통일

4. 적용 범위

- 파트너 등록, 계약 정보, 정산 계좌 입력 화면에 같은 검증 규칙을 점진 적용
- 신규 폼 화면 리뷰 시 QA가 동일한 상태 이름으로 이슈를 남길 수 있게 정리

5. 면접에서 설명할 판단

- 검증 로직과 UI 메시지 매핑을 분리해 폼 화면이 늘어나도 규칙을 재사용하도록 만든 과정
- 사용자가 오류를 복구할 수 있도록 필드 에러와 서버 에러의 위치를 다르게 둔 이유

Example Product JD 연결 요약

Dashboard UI

OpsPulse에서 운영자가 지표, 필터, 추세를 한 화면에서 비교하도록 리포트 화면을 설계했습니다.

API State

TanStack Query와 URL query를 분리해 서버 상태와 화면 조건을 재현 가능하게 만들었습니다.

Admin Workflow

ReviewFlow에서 검토 큐, 상세 패널, 액션 버튼 상태를 나눠 운영자 처리 흐름을 단순화했습니다.

Form Quality

FormGuard에서 Zod 스키마와 에러 메시지 매핑으로 입력 단계의 QA 이슈를 줄였습니다.

1. 지원 직무와 직접 연결되는 근거

- React/TypeScript 기반 업무용 화면 구현 경험
- 대시보드, 테이블, 필터, 폼 검증 등 B2B SaaS 화면 패턴 경험
- API 응답, 서버 상태, URL 상태를 분리한 화면 상태 설계 경험

2. 면접에서 확장할 수 있는 질문

- 왜 URL query를 필터 상태의 기준으로 삼았는가
- 폼 검증을 Zod 스키마와 UI 메시지 매핑으로 분리한 이유는 무엇인가
- 운영자 화면에서 빈 상태, 권한 없음, 서버 에러를 나눈 기준은 무엇인가

3. 프로젝트별 강조 포인트

- OpsPulse: 대시보드 필터 재현성과 운영 리포트 흐름 개선
- ReviewFlow: 검토 큐, 상세 패널, 액션 상태 분리
- FormGuard: 입력 검증, 에러 메시지 우선순위, QA 재현성

4. 보여줄 수 있는 작업 방식

- 요구사항을 API 상태, 화면 상태, 사용자 액션, 예외 상태로 나누어 구현 범위를 정리
- 기능 구현을 단순 목록으로 두지 않고 사용자/운영자 경험 개선 결과와 함께 설명